

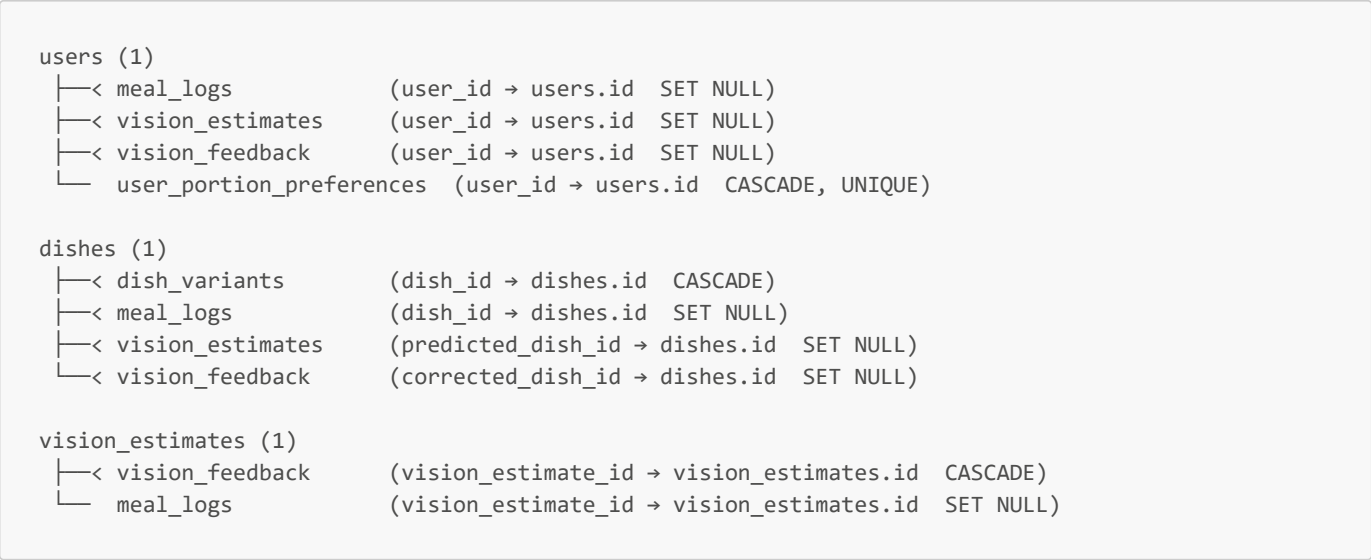
Database Breakdown — NutriLabelAI

Engine: PostgreSQL 16 + pgvector extension
ORM: SQLAlchemy 2.0 (mapped_column / Mapped syntax)
Migrations: Alembic ([alembic/versions/](#))
Connection string: postgresql+psycopg://postgres:postgres@db:5432/nutrition
Docker service: nutrition_db (image: pgvector/pgvector:pg16)

Table of Contents

- 1. [Overview / ER Diagram](#)
- 2. [Table Details](#)
 - [users](#)
 - [dishes](#)
 - [dish_variants](#)
 - [vision_estimates](#)
 - [meal_logs](#)
 - [vision_feedback](#)
 - [user_portion_preferences](#)
- 3. [Indexes](#)
- 4. [Key Design Decisions](#)
- 5. [Semantic Search Query](#)
- 6. [Migrations](#)
- 7. [Data Sources](#)

1. Overview / ER Diagram



Creation order (FK dependency chain): [users](#) → [dishes](#) → [dish_variants](#) → [vision_estimates](#) → [meal_logs](#) → [vision_feedback](#) → [user_portion_preferences](#)

2. Table Details

2.1 [users](#)

Identity anchor. No authentication — identified solely by a device UUID generated on first install.

Column	Type	Constraints	Notes
id	BIGSERIAL	PK	Auto-increment
device_id	VARCHAR(255)	NOT NULL, UNIQUE	Mobile UUID from first launch
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL, auto-trigger	Updated by <code>_update_updated_at</code> trigger

Relationships: one-to-many with `meal_logs`, `vision_estimates`, `vision_feedback`; one-to-one with `user_portion_preferences`.

2.2 dishes

Canonical nutrition knowledge base. ~2,400,000 rows. All nutrient values are **per 100 g** unless `serving_size_g` overrides that reference.

Column	Type	Nullable	Notes
id	BIGSERIAL	PK	
name	VARCHAR(255)	NOT NULL	Canonical dish name
Core macros			
calories	NUMERIC(8,2)	NOT NULL, ≥ 0	kcal per 100 g
protein_g	NUMERIC(8,2)	NOT NULL, ≥ 0	
fat_g	NUMERIC(8,2)	NOT NULL, ≥ 0	Total fat
carbs_g	NUMERIC(8,2)	NOT NULL, ≥ 0	Total carbohydrates
FDA micronutrients			All nullable — source data may be missing
fiber_g	NUMERIC(8,2)	nullable	
sugar_g	NUMERIC(8,2)	nullable	
sodium_mg	NUMERIC(8,2)	nullable	
potassium_mg	NUMERIC(8,2)	nullable	
saturated_fat_g	NUMERIC(8,2)	nullable	
trans_fat_g	NUMERIC(8,2)	nullable	
cholesterol_mg	NUMERIC(8,2)	nullable	
vitamin_a_mcg	NUMERIC(8,2)	nullable	
vitamin_c_mg	NUMERIC(8,2)	nullable	
vitamin_d_mcg	NUMERIC(8,2)	nullable	
calcium_mg	NUMERIC(8,2)	nullable	
iron_mg	NUMERIC(8,2)	nullable	
Serving reference			
serving_size_g	NUMERIC(8,2)	nullable, > 0	If present, overrides the per-100 g baseline
serving_size_unit	VARCHAR(50)	default 'g'	
Metadata			
category_name	VARCHAR(100)	nullable	e.g. 'Grains', 'Poultry'

Column	Type	Nullable	Notes
data_source	VARCHAR(100)	nullable	'USDA', 'FastFood', 'Manual', etc.
confidence_score	NUMERIC(4,3)	nullable, 0–1	Source-data quality indicator
is_active	BOOLEAN	NOT NULL, default TRUE	Soft-delete flag
version	INTEGER	NOT NULL, default 1	For future model-update versioning
created_at / updated_at	TIMESTAMP TZ		updated_at auto-maintained by trigger

Check constraints: calories ≥ 0, protein_g ≥ 0, fat_g ≥ 0, carbs_g ≥ 0

2.3 dish_variants

Searchable text aliases with **384-dimensional vector embeddings** for semantic search. Every dish has at least one variant (its canonical name); many have additional aliases (regional names, common misspellings, translations).

~2,400,000 rows — roughly one-to-one with dishes in practice.

Column	Type	Nullable	Notes
id	BIGSERIAL	PK	
dish_id	BIGINT	NOT NULL, FK → dishes.id CASCADE	
variant_text	TEXT	NOT NULL	The alias string (what users type/say)
embedding	VECTOR(384)	NOT NULL	Generated by all-MiniLM-L6-v2
variant_type	VARCHAR(50)	nullable	'canonical' / 'alias' / 'regional' / 'user_generated' / 'misspelling'
language_code	VARCHAR(10)	NOT NULL, default 'en'	ISO 639-1
search_count	INTEGER	NOT NULL, default 0	Hit tracking
last_searched_at	TIMESTAMP TZ	nullable	Updated on each search hit
created_at / updated_at	TIMESTAMP TZ		

Unique constraint: (dish_id, variant_text) — no duplicate alias per dish.

HNSW index (the performance-critical index):

```
CREATE INDEX idx_dish_variants_embedding_hnsw
ON dish_variants
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

- Algorithm: Hierarchical Navigable Small World graph
- Distance metric: cosine (<=> operator)
- Parameters: m=16 (connectivity), ef_construction=64 (build quality)
- Enables sub-100 ms ANN search across 2.4 M vectors

2.4 vision_estimates

Records every camera-pipeline prediction. Images are **never stored as base64** — only storage path/key references are kept. Enables future reprocessing when models are updated.

Column	Type	Notes
id	BIGSERIAL PK	
user_id	BIGINT FK → users SET NULL	Nullable — anonymous sessions allowed
capture_mode	VARCHAR(20) CHECK	'depth' / 'multi_angle' / 'single'
device_type	VARCHAR(30) nullable	'ios', 'android', 'ios_lidar', etc.
num_images	INTEGER NOT NULL	Count of frames sent
has_depth_data	BOOLEAN default FALSE	LiDAR depth available
image_storage_keys	JSONB nullable	["img/uuid1.jpg", ...] — S3/local paths
depth_storage_key	VARCHAR(500) nullable	Path to depth map
predicted_dish_id	BIGINT FK → dishes SET NULL	Top match (nullable so history survives dish deletion)
predicted_dish_name	VARCHAR(255) NOT NULL	Denormalised — preserved if dish is deleted
predicted_confidence	NUMERIC(4,3) nullable	0–1
alternative_dishes	JSONB nullable	[{dish_id, dish_name, confidence}, ...]
estimation_mode	VARCHAR(20) NOT NULL	Pipeline mode string
volume_ml	NUMERIC(8,2) nullable	Estimated food volume
volume_confidence	NUMERIC(4,3) nullable	
calorie_estimate	NUMERIC(8,2) NOT NULL, ≥ 0	Primary output
calorie_range_min/max	NUMERIC(8,2) nullable	Uncertainty bounds
classifier_version	VARCHAR(50) nullable	Model version tag
segmentation_version	VARCHAR(50) nullable	
volume_estimator_version	VARCHAR(50) nullable	
processing_time_ms	INTEGER nullable	End-to-end pipeline latency
created_at	TIMESTAMP TZ NOT NULL	

2.5 meal_logs

Single source of truth for confirmed meals. Both the manual text-search path and the camera path converge here. Stores a **frozen JSONB snapshot** of the full FDA nutrition label at the time of logging — future model updates never alter historical records.

Column	Type	Notes
id	BIGSERIAL PK	
user_id	BIGINT FK → users SET NULL	
dish_id	BIGINT FK → dishes SET NULL	Nullable — log survives dish deactivation
vision_estimate_id	BIGINT FK → vision_estimates SET NULL	NULL for manual entries
entry_source	VARCHAR(20) CHECK	'manual' / 'camera'
logged_dish_name	VARCHAR(255) NOT NULL	Denormalised display name
logged_calories	NUMERIC(8,2) NOT NULL, ≥ 0	Calories at time of log
nutrition_label	JSONB nullable	Full FDA label snapshot (frozen forever)
serving_multiplier	NUMERIC(6,3) default 1.0	e.g. 0.5 = half portion, 2.0 = double

Column	Type	Notes
match_confidence	NUMERIC(4,3) nullable	Similarity score from retrieval
model_version	VARCHAR(50) nullable	Model used for this estimation
logged_at	TIMESTAMPTZ NOT NULL	When the user confirmed the meal
created_at	TIMESTAMPTZ NOT NULL	Row insertion time

Why denormalise `logged_dish_name` and `logged_calories`?

If a dish is later soft-deleted or its nutrition data is corrected, the user's historical log must remain accurate. These fields are immutable snapshots.

2.6 vision_feedback

User corrections on camera estimates. Drives both model improvement and per-user personalisation.

Column	Type	Notes
id	BIGSERIAL PK	
vision_estimate_id	BIGINT FK → <code>vision_estimates</code> SET NULL	
user_id	BIGINT FK → <code>users</code> SET NULL	
corrected_dish_id	BIGINT FK → <code>dishes</code> SET NULL	What the user said it actually was
feedback_type	VARCHAR(30) CHECK	'confirmed' / 'corrected_dish' / 'corrected_portion' / 'quick_correction'
confirmed_dish_name	VARCHAR(255) nullable	Free text correction
portion_adjustment	NUMERIC(6,3) nullable	Multiplier: 0.5 = half, 2.0 = double
plate_size	VARCHAR(30) nullable	Reference plate size used
quick_feedback	VARCHAR(20) nullable	'accurate' / 'too_high' / 'too_low' / 'wrong_dish'
corrected_calories	NUMERIC(8,2) nullable	User's stated actual calories
notes	TEXT nullable	Free text notes
created_at	TIMESTAMPTZ NOT NULL	

2.7 user_portion_preferences

Aggregated personalisation profile — one row per user (UNIQUE on `user_id`). Built from the user's feedback history by `vision_feedback_service.py`.

Column	Type	Notes
id	BIGSERIAL PK	
user_id	BIGINT NOT NULL, UNIQUE, FK → <code>users</code> CASCADE	One profile per user
avg_portion_factor	NUMERIC(6,3) default 1.0	Global serving-size bias
feedback_count	INTEGER default 0	Number of feedback events used
confidence_score	NUMERIC(4,3) default 0.0	How reliable this profile is
dish_preferences	JSONB nullable	{"dish_id": {"avg_factor": 1.2, "count": 5}}

Column	Type	Notes
category_preferences	JSONB nullable	{"Grains": {"avg_factor": 0.9, "count": 12}}
last_updated	TIMESTAMPTZ	Auto-updated on write
created_at	TIMESTAMPTZ	

3. Indexes

Index	Table	Columns	Type	Purpose
idx_users_device_id	users	device_id	BTree	Fast lookup on app launch
idx_dishes_name	dishes	name	BTree	Name search / admin queries
idx_dishes_data_source	dishes	data_source	BTree	Filter by source (USDA, FastFood...)
idx_dishes_active	dishes	is_active WHERE TRUE	Partial BTree	Filters inactive rows in every query
idx_dish_variants_dish_id	dish_variants	dish_id	BTree	JOIN to dishes
idx_dish_variants_type	dish_variants	variant_type	BTree	Filter by alias type
idx_dish_variants_variant_text	dish_variants	variant_text	BTree	Exact text lookup
idx_dish_variants_embedding_hnsw	dish_variants	embedding	HNSW cosine	Semantic ANN search ⭐
idx_vision_estimates_user_id	vision_estimates	user_id	BTree	Per-user history
idx_vision_estimates_created_at	vision_estimates	created_at	BTree	Time-range queries
idx_vision_estimates_dish_id	vision_estimates	predicted_dish_id	BTree	Per-dish analytics
idx_meal_logs_user_id	meal_logs	user_id	BTree	Fetch user history
idx_meal_logs_logged_at	meal_logs	logged_at	BTree	Time-range / "today's entries"
idx_meal_logs_dish_id	meal_logs	dish_id	BTree	Per-dish usage stats
idx_vision_feedback_estimate_id	vision_feedback	vision_estimate_id	BTree	Feedback per estimate
idx_vision_feedback_user_id	vision_feedback	user_id	BTree	Per-user feedback

Index	Table	Columns	Type	Purpose
				history
<code>idx_user_portion_preferences_user_id</code>	<code>user_portion_preferences</code>	<code>user_id</code>	BTree	Personalisation lookup

4. Key Design Decisions

Soft-delete instead of hard-delete on `dishes`

`is_active = FALSE` deactivates a dish without breaking existing foreign keys in `meal_logs`, `vision_estimates`, or `vision_feedback`. The partial index `WHERE is_active = TRUE` keeps all queries fast.

SET NULL foreign keys (not CASCADE on most tables)

All FKs that point from transactional tables (`meal_logs`, `vision_estimates`, `vision_feedback`) back to `users` or `dishes` use `ON DELETE SET NULL`. This preserves audit history even when a user is deleted or a dish is removed.

Frozen `nutrition_label` JSONB snapshot in `meal_logs`

The full FDA label is serialized and stored at log time. This means a user's history is never retroactively changed by model updates or dish corrections. The denormalised `logged_dish_name` and `logged_calories` columns serve the same purpose for display.

No base64 in `vision_estimates`

`image_storage_keys` stores only file paths / object-store keys. Storing raw image bytes in the DB would bloat it significantly and prevent efficient re-processing.

`_update_updated_at` trigger (shared function)

A single PL/pgSQL function is created once and reused by triggers on `users`, `dishes`, and `dish_variants` to keep `updated_at` accurate without application-layer involvement.

HNSW parameters (`m=16`, `ef_construction=64`)

These are conservative defaults chosen for a balance of build speed and query accuracy on the 2.4 M-row dataset. `m=16` gives each node 16 bi-directional links in the graph; `ef_construction=64` controls how many candidates are evaluated during index build. Query accuracy (`ef_search`) is controlled at query time by the client.

5. Semantic Search Query

The core retrieval path in `retrieval_service.py`:

```
SELECT
  d.id,
  d.name,
  d.category_name,
  d.calories,
  COALESCE(d.protein_g, 0.0) AS protein_g,
  COALESCE(d.carbs_g, 0.0) AS carbs_g,
  COALESCE(d.fat_g, 0.0) AS fat_g,
  d.fiber_g, d.sugar_g, d.sodium_mg, d.potassium_mg,
  d.saturated_fat_g, d.trans_fat_g, d.cholesterol_mg,
  d.vitamin_a_mcg, d.vitamin_c_mg, d.vitamin_d_mcg,
  d.calcium_mg, d.iron_mg,
  dv.variant_text,
  1 - (dv.embedding <=> CAST(:query_vector AS vector)) AS similarity
FROM dish_variants dv
JOIN dishes d ON dv.dish_id = d.id
```

```
WHERE d.is_active = TRUE
AND (1 - (dv.embedding <=> CAST(:query_vector AS vector))) >= :threshold
ORDER BY dv.embedding <=> CAST(:query_vector AS vector)
LIMIT :k;
```

Parameter	Default	Description
:query_vector	—	384-dim float array from <code>all-MiniLM-L6-v2</code>
:threshold	0.45	Minimum cosine similarity (configured in <code>settings.py</code>)
:k	5	Top-k results (configured in <code>settings.py</code>)

The `<=>` operator is **cosine distance** (0 = identical). Similarity = `1 - distance`. Results are deduplicated by `dish_id` in Python in case multiple variants of the same dish all match.

6. Migrations

File	Description
<code>alembic/versions/0001_init_schema_with_pgvector.py</code>	Creates all 7 tables, installs <code>pgvector</code> extension, creates the HNSW index, drops any legacy tables first, installs <code>_update_updated_at</code> trigger on <code>users</code> , <code>dishes</code> , <code>dish_variants</code>
<code>alembic/versions/0002_add_vision_feedback_schema.py</code>	No-op placeholder (<code>0002_noop</code>) — all tables live in 0001. Keeps the revision chain intact for future migrations.

To apply migrations from scratch:

```
docker-compose up -d db
docker exec -it nutrition_api alembic upgrade head
```

To check current revision:

```
docker exec -it nutrition_api alembic current
```

7. Data Sources

The `dishes.data_source` column tracks provenance. Current values in production:

Source	Approx. rows	Description
USDA	~2,400,000	USDA FoodData Central — branded + foundation + SR Legacy + survey foods
FastFood	~515	Scraped fast food items (<code>data/fastfood.csv</code>)
Manual / seed	~1	Hand-entered seed dishes (<code>data/seed_dishes.csv</code>)

Raw data files:

- `data/usda_branded_foods.csv` — Full USDA branded foods export
- `data/usda_branded_foods_reduced.csv` — Reduced subset used in ingestion
- `data/fastfood.csv` — Fast food nutritional data
- `data/seed_dishes.csv` — Manual seed entries
- `data/foundation/` — USDA foundation foods

- [data/sr_legacy/](#) — USDA SR Legacy foods
- [data/survey/](#) — USDA FNDDS survey foods

Ingestion scripts (in [scripts/](#)): [import_usda_dishes.py](#), [import_fastfood.py](#), [ingest_seed.py](#), [import_usda_fixed.py](#), [import_usda_whole_foods.py](#)